COM4521/COM6521 Parallel Computing with Graphical Processing Units (GPUs)

Assignment (80% of module mark)

Deadline: 5pm Thursday 21st May (Week 12)

Starting Code: *Download Here*

Document Changes

Any corrections or changes to this document will be noted here and an update will be sent out via the course's Google group mailing list.

Document Built On: 12 May 2026

Introduction

This assessment has been designed against the module's learning objectives. The assignment is worth 80% of the total module mark. The aim of the assignment is to assess your ability and understanding of implementing and optimising parallel algorithms using both OpenMP and CUDA.

An existing project containing a single threaded implementation of three algorithms has been provided, this starting code should not be considered highly optimised. This provided starting code also contains functionality for validating the correctness, and timing the performance of your implemented algorithms.

You are expected to implement both an OpenMP and a CUDA version of each of the provided algorithms, and to produce a report to document and justify the techniques you have used, and demonstrate how profiling and/or benchmarking supports your justification.

The Algorithms & Starting Code

Three algorithms have been selected which cover a variety of parallel patterns for you to implement. As these are independent algorithms, they can be approached in any order and their difficulty does vary. You may redesign the algorithms in

your own implementations for improved performance, providing input/output pairs remain unchanged.

The reference implementation and starting code are available to download from here (and blackboard).

Each of the algorithms are described in more detail below.

Count Gliders

You are provided three parameters:

- `cells`, an array of `unsigned char` values, which represents a frame from Conway's Game of Life).
- `width`, the width of array `cells`.
- `height`, the height of array `cells`.

You must count, and return, the number of gliders present.

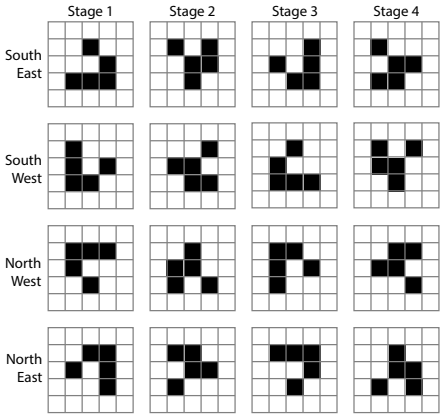

Figure 1: A table showing the four glider stages, in each of the four possible directions. They are effectively made up of two patterns, which are reflected and rotated.

As shown in Figure 1 gliders have four unique stages, which can each be rotated in four directions.

A valid glider match requires a 3x3 area of the image to match one of the sixteen glider patterns shown in Figure 1.

The reference algorithm can be found within `cpu.c::cpu_countGliders()`.

It can be executed via specifying the path to a `.png` image, where all black (or 0 in the red channel) pixels will be treated as empty.

An example input file matching `cg_16_in.png`, containing all 16 glider forms from Figure 1, has been provided in the handout code.

Index	0	1	2	3	4	5	6	7	8	9	10	11	12
Numbers	77	32	18	26	89	98	43	27	105	9	82	50	28
Bin Index	7	3	1	2	8	9	4	2	10	0	8	5	2
Histogram	1	1	3	1	1	1	0	1	2	1	1	0	0

Table 1: Calculation of the histogram, where `bin_width==10`, of the array in row "Numbers". Row "Bin Index" shows each numbers' corresponding bin index, and row "Histogram" shows the final histogram that would be returned, by counting the occurrence of each bin's index in row "Bin Index".

```
<executable> CPU CG cg_16_in.png
```

Histogram

Thrust/CUB may not be used in your code for this stage of the assignment.

You are provided four parameters:

- `numbers`, an array of `int` values in the inclusive range `[0, 255]`.
- `length`, the length of array `numbers`
- `bin_width`, the width of the histogram bins.
- `histogram_out`, a pre-allocated host array to store the resulting histogram.

You must generate the corresponding histogram of `numbers` and store it within `histogram_out`. An example histogram calculation is show in Table 1.

`bin_width` is such that a bin width of `10` would mean the inclusive range `[0, 9]` are assigned to bin `0`, inclusive range `[10, 19]` to bin `1` and so forth.

The reference algorithm can be found within `cpu.c::cpu_histogram()`.

It can be executed either via specifying a random seed, `length` and `bin_width`.

```
<executable> CPU H 12 100000 10
```

Or via specifying the path to a `.csv` input file (1 number per row) and `bin_width`, e.g.:

```
<executable> CPU H histogram_in.csv
```

Emboss

You are provided four parameters:

- `pixels`, an array of `unsigned char` values (cells), which represents a pixels of an 3-channel RGB image.
- `width`, the width of image held in `pixels`.
- `height`, the height of image held in `pixels`.
- `output`, an array of `unsigned char` values `cells`, which represents a pixels of an 1-channel RGB image.

The emboss operator shown in Equation 1 should be applied to the image, to produce an emboss effect.

$$\begin{bmatrix} -2 & -1 & 0 \\ -1 & 0 & 1 \\ 0 & 1 & 2 \end{bmatrix} \tag{1}$$

A emboss operator is applied by aligning the centre of the operator with a pixel, and summing the result of multiplying each weight with it's corresponding pixel. The resulting value must then be normalised (+128) and clamped, to ensure it does not go out of bounds. This produces an image

Figure 2: An example input using a photo of Regent Court (left) and the embossed output (right).

Pixels at the edge of the image cannot have the image kernel applied, as they lack all neighbouring pixels, and therefore cannot be processed. As such, `output` image will be 2 pixels smaller in each dimension. Similarly, the output image is greyscale, so only has 1-channel.

The reference algorithm can be found within `cpu.c::cpu_emboss()`.

It can be executed via specifying the path to an input `.png` image, optionally a second output `.png` image to store the result can be specified, e.g.:

```
<executable> CPU E e_in.png e_out.png
```

The Task

Code

For this assignment you must complete the code found in both `openmp.c` and `cuda.cu`, so that they perform the same algorithm described above and found in the reference implementation (`cpu.c`), using OpenMP and

CUDA respectively. You should not modify or create any other files within the project. The two algorithms to be implemented are separated into 3 methods named `openmp_countGliders()`, `openmp_histogram()` and `openmp_chromaticaberration()` respectively (and likewise for CUDA).

You should implement the OpenMP and CUDA algorithms with the intention of achieving the fastest performance for each algorithm.

It is important to free all allocated memory as memory leaks could cause the benchmark mode, which repeats the algorithm, to run out of memory during marking.

Report

You are expected to provide a technical report alongside your code submission. For each of the two implementations (OpenMP and CUDA) you should detail:

- A clear description of your approach.
- A justification for your approach.
- Limitations of your approach and how they were identified.
- How it compares to the other implementations.

The proportion of your report dedicated to CUDA, is expected to be longer, as CUDA is a greater proportion of the module's taught content.

The report should be no longer than 3000 words, and you are encouraged to include relevant figures.

Benchmarks should **always be carried out in Release mode**, with timing averaged over several runs. The provided project code has a runtime argument `--bench` which will repeat the algorithm for a given input 100 times (defined in `config.h`). It is important to benchmark over a range of inputs, to allow consideration of how the performance of each algorithm scales.

Deliverables

You must submit your `openmp.c`, `cuda.cu` and your report document (e.g. `.pdf`/`.docx`) within a single zip file via Blackboard, before the deadline. Your code should build in the Release mode configuration without errors or warnings (other than those caused by IntelliSense) on university managed desktops. You do not need to hand in any other project or code files other than `openmp.c`, `cuda.cu`. As such, it is important that you do not modify any of the other files provided in the starting code so that your submitted code remains compatible with the projects that will be used to mark your submission.

Your code should not rely on any third party tools/libraries **except for those** introduced within the lectures/lab classes. Hence, the use of Thrust and CUB is permitted **except for when implementing the Histogram algorithm**.

Even if you do not complete all aspects of the assignment, partial progress should be submitted as this can still receive marks.

Marking

Your submission will be assessed primarily based on your written report. However, up to 30% of this mark may be deducted if the submitted implementations fail to pass a range of correctness tests.

Marks will be awarded according to how effectively your submission demonstrates understanding of the module's learning objectives:

- Compare and contrast different parallel computing architectures
- Implement programs for both GPU and multicore (CPU) architectures
- Apply benchmarking and profiling techniques to evaluate GPU program performance
- Identify performance bottlenecks and apply optimisations to improve code efficiency

Assessment will follow these criteria:

- **Terminology** – Is the explanation clear and precise, avoiding over-reliance on source code to convey the approach?
- **Knowledge** – Does the report demonstrate awareness of appropriate techniques for parallelising the algorithm and evaluating its performance?
- **Understanding** – Is there evidence of a sound grasp of the theoretical concepts that support the chosen implementation strategies?
- **Communication** – Is the report well-structured, coherent, and easy to follow?
- **Correctness** – Do the submitted implementations produce the expected results?

If you submit work after the deadline you will incur a deduction of 5% of the mark for each working day that the work is late after the deadline. Work submitted more than 5 working days late will be graded as 0. This is the same lateness policy applied university wide to all undergraduate and postgraduate programmes.

Assignment Help & Feedback

The lab classes should be used for feedback from demonstrators and the module leaders. You should aim to work iteratively by seeking feedback throughout the semester. If leave your assignment work until the final week you will limit your opportunity for feedback.

For questions you should either bring these to the lab classes or use the course's Google group (COM4521-group@sheffield.ac.uk) which is monitored by the

course's teaching staff. However, as messages to the Google group are public to all students, emails should avoid including assignment code, instead they should be questions about ideas, techniques and specific error messages rather than requests to fix code.

If you are uncomfortable asking questions, you may prefer to use the course's anonymous google form. Anonymous questions must be well formed, as there is no possibility for clarification, otherwise they risk being ignored.

Please do not email teaching assistants or the module leader directly for assignment help. Any direct requests for help will be redirected to the above mechanisms for obtaining help and support.